Change Data Feed

Change Data Feed

Delta log tracks or deals with file or dataset level changes.

We haven't understood about Row level changes (from this particular file, ignore the following Row)

CDF is used to track the row level changes between different versions. (All the operations that happened from one version to another)

Operations that can happen at Row Level are -

1. Inserts
2. Deletes
3. Updates

    Pre-image : The status of the Row before update

    Post-image : The status of the Row after update.

Why is this needed ? For Granular level audit. If we want to provide only the latest data for the downstream teams (Incremental processing - Process only the latest Data)

Demo :

Step-1 : Creating a Table called demo

```
CREATE TABLE demo (
    id INT,
    name STRING,
    amount DOUBLE
);
```

[By default it is a Delta Table]

Step-2 : Insert records into the Table

%sql

INSERT INTO demo VALUES (1, ‘alice’, 100.0), (2, ‘bob’, 200.0), (3, ‘carol’, 300.0);

Step-3 : Enable Change Data Feed

To track Row Level Changes, enable ‘Change Data Feed’ feature.

This can be done at two levels

1. While table creation
2. By Altering the existing table

   %sql

   ALTER TABLE demo SET TBLPROPERTIES (delta.enablechangedatafeed = true);

Perform a set of operations after enabling the change data feed -

%sql

–Update

UPDATE demo SET amount = amount * 1.1 WHERE id = 2;

–Insert additional

INSERT INTO demo VALUES (4, ‘dan’, 400.0);

–delete

DELETE FROM demo WHERE id = 1;

[describe history demo; (command to view the versions)]

After performing the above operations, there will be 5 entries

1. Update
2. Write (for insert)
3. Optimize (automatically optimized)
4. Delete
5. Optimize (automatically optimized)

%sql

SELECT * FROM table_changes (‘demo’, 2)

(Granular level changes displayed

- This displays all the row level changes that have happened after version 2.
- There will be pre-image and post-image entries for the Update operation
- This doesn't display the granular level changes entries for version 1 where the change data feed was not enabled.

)

Example Scenario - Downstream team requests for all the deleted records.

```
CREATE TABLE IF NOT EXISTS demo_deleted_data_batch (
    Id INT,
    name STRING,
    amount DOUBLE,
    _change_type STRING,
    _commit_version LONG,
    _commit_timestamp TIMESTAMP
)
USING delta;
```

Batch Approach

```
%sql

INSERT INTO demo_deleted_data_batch

SELECT id, name, amount, _change_type, _commit_version,
_commit_timestamp

FROM table_changes (‘demo’, 2)

WHERE _change_type = ‘delete’;
```

Streaming Approach

Instead of taking as a batch, the data needs to be read as a stream of continuous data

```
from spark.sql.functions import col

( spark.readStream.format(“delta”)

        .option(“readChangeFeed”, “true”)

        .option(“startingVersion”, 2)

        .table(“demo”)

        .filter(col(“change_type”) == “delete”)

        .select(“id”,”name”)

        .writeStream

        .outputMode(“append”)

        .option(“checkpointLocation”, “/volumes/…”)

        .trigger(availableNow = True)

        .table(“demo_deleted_data_streaming)
```

[Whenever streaming, we need to provide the check point location, so that it can remember whatever is processed earlier and it can process incrementally

- trigger(availableNow=True) indicates -> Run, process the data and stop it. No need to run it continuously

]

Advantage of Streaming Approach -

- In the case of a batch, if you want the data to be updated incrementally, then we need to develop our own logic. By default, in batch there will be duplicates and needs processing from the beginning again.
- In the case of streaming, the operations performed earlier are stored in the checkpoint location. Since all of the activities are saved, the next processing starts from the point where it was left earlier and doesn't have to re-do everything from the beginning again.

Note :

- In case of serverless, the microbatches style of processing is not supported. For example, if we use : trigger(processingTime="2seconds"), this is not supported in serverless. We need to start a compute cluster to execute the above.
- ProcessingTime = "2seconds" implies, treat it like a microbatch of 2 secs, i.e., run every 2 secs.
- Whenever there are changes in the source table, it will be reflected to the downstream team as well in case of streaming approach.

Enabling the Change Data Feed while table creation :

```
CREATE TABLE demo (
    id INT,
    name STRING,
    amount DOUBLE
)
USING DELTA
TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

Enabling the Change Data Feed at Spark Session level by setting the property:

SET spark.databricks.delta.properties.defaults.enableChangeDataFeed = true;

(This works for classic compute cluster and not for serverless compute)

Properties

availableNow = True

- Process all the data that is currently available and stop. It runs as a finite job (like a batch job) and stops automatically. In case of batch processing, we have to implement incremental logic. But in the case of availableNow, it is automatically done.
- If it is re-run later with the same checkpoint, it will pick up only the new changes.
- When is best to use availableNow? Best for serverless as continuous triggers are not supported in serverless.
- If there is no requirement to analyse it in real-time. If it is required on a daily or hourly basis, then periodic jobs can be scheduled. Schedule a Notebook as a job, this doesn't require the cluster to be alive always.

Shallow Cloning

Cloning

Cloning is creating a copy of the delta table at whichever version is required.

Types of Clones

1. Shallow Clone - In this case only the metadata is copied. The copied table points or references to the source data.
2. Deep Clone - In this case, both the metadata and data is copied.

Demo Steps :

1. Connect to the catalog

   %sql

   use catalog deltalake_catalog;

2. Create a new table named orders_managed (create managed table)

   %sql

   DROP TABLE IF EXISTS orders_managed;

   CREATE OR REPLACE TABLE orders_managed (

   order_id BIGINT,

   sku STRING,

   product_name STRING,

   product_category STRING,

   qty INT,

   unit_price DECIMAL(10,2)

   );

3. Alter the table by adding a check constraint

   ALTER TABLE orders_managed ADD CONSTRAINT valid_qty CHECK (qty > 0);

4. Insert records

   New parquet and Json file will be created on inserting the records

Insert few more additional rows

5. Delete a record

   Deletion vector will be created and Optimize will run. Where everything will be consolidated in one parquet.

6. Update a record

   One Deletion vector will be created and when Optimize would have run, a consolidated parquet file will be created

[Optimize happens automatically after certain activities like Update and Delete.]

(To Check the dependent files)

%sql

describe detail <table_name>

Performing a Shallow Clone on the table -

%sql

DROP TABLE IF EXISTS orders_managed_sclone

CREATE OR REPLACE TABLE orders_managed_sclone SHALLOW CLONE orders_managed;

DESCRIBE DETAIL orders_managed_sclone;

(By Default, if we just mention CLONE, then it will perform a Deep Clone)

Note :

- Only the Metadata and for the latest 2 versions will be copied. It refers to the 2 latest files of the source table.
- Both the source and the shallow cloned tables will have the exact same data present as it is referring to the same parquet files of the source table.

Example Scenarios :

- Inserting records in the source table (Will it reflect in the cloned table?)

  INSERT INTO orders_managed VALUES

  (11, ‘SKU-1101’, ‘Resistance Bands Set’, ‘Fitness’, 1, 899.00),

  (12, ‘SKU-1102’, ‘Sticky Notes Pack’, ‘Stationary’, 6, 59.00)

  %sql

  select * from orders_managed_sclone order by order_id

  [This doesn’t modify the cloned table. As the copy of the table was done at a particular version of the source table. The changes made to the source table will not be reflecting the cloned table]

- Inserting the records in the Cloned tables (Will it show up in the source?) No, there will be no changes in the source table. **There are 2 files now, one of the parquet files is of the source which the cloned table had referenced and the second file is a newly created file of the cloned table.**
- In the case of shallow clone, the source and cloned tables are not dependent on each other.
- Deleting the records from the cloned table will create a new file, followed by an auto optimize activity with one consolidated parquet file.
- What if all the records from the source table are deleted and perform an aggressive like VACUUM retain 0 hours? (Ideally all files should be deleted)

  The 2 files which the shallow cloned table is referencing are not deleted. However, the rest of the files are removed on a Vacuum.

- Deleting the records from the cloned table :

  %sql

  DELETE FROM orders_managed_sclone;

- Vacuum Dry Run on orders Managed :

  %sql

  ALTER TABLE orders_managed_sclone

  SET TBLPROPERTIES (

  ‘delta.deletedFileRetentionDuration’ = ‘interval 0 hours’

)

  %sql

  VACUUM order_managed_sclone DRY RUN;

  (This should display totally 5 files, 2 from source and the remaining from the cloned table)

- Restoring the table to a specific version

  %sql

  restore table orders_managed version as of 8;

- Vacuum would work only if the files are not referenced from source / cloned tables.
-

Deep Cloning

Deep Clone

Deep clone copies both the Data and the Meta data. That is why it is storage and time consuming and also process heavy. It is a heavy operation as compared to the Shallow Clone.

Example Scenario :

- Creating Source Managed orders Table and performing some operations like inserts, updates, etc.
- Creating a Deep Clone Table.

  %sql

  DROP TABLE IF EXISTS orders_managed_dclone;

  CREATE OR REPLACE TABLE orders_managed_dclone DEEP CLONE orders_managed;

  DESCRIBE DETAIL orders_managed_dclone;

  (By Default it is Deep Clone even if the DEEP Keyword is not mentioned in the query.)

- Deep Clone doesn't copy all the Data, but rather copies the required data of the latest versions from where it was cloned.

  %sql

  select * from orders_managed order by order_id

  %sql

  SELECT DISTINCT _metadata.file_path FROM orders_managed_dclone;

- Inserting the records in the source table, will it reflect in the cloned table?

  Any changes to the source table don't impact the cloned data and vice-versa.

- Deleting the data from the source managed table and altering the table for performing VACUUM, this will delete the records from the source

table but doesn’t impact the deep cloned table as it has its own copy of data.

- Any changes made to either the Deep or Shallow Clone, affects only the clone themselves and not the source table.
- Cloning the table is not the same as CTAS (Create Table As Select *)
- Deep Clones can be used to preserve the state of the table at a certain point in time for archival purposes.

  To take a snapshot of the production table every month -

  create or replace table archive_table CLONE my_prod_table;

- In order to test a workflow on a prod table without corrupting the prod table, a shallow clone can be created. This will create a separate copy of the production table and will not impact the original production table. This allows to run arbitrary workflows on the cloned table that contains all the production data but doesn’t affect the production workloads.
- Shallow clones on external tables should be external and the shallow clones on managed tables should be managed tables.
- For managed tables, dropping the source table breaks the target table for shallow clones. (But this data is available for the duration of 7 days)
- Shallow clones on the external table are not impacted by dropping the source.
- Clones can be taken for any version of the table.

DeltaLake UniForm

DeltaLake UniForm

Delta is a very famous lakehouse file format.

An equivalently famous file format is Iceberg.

The common factor between them is that they are stored as parquet files. The only difference between Delta and Iceberg is how they maintain and structure the Metadata otherwise the data is the same underneath.

- Say for example, we have a Delta Table and a client wants to read the data in the Iceberg format, then they won't be able to read as the format is different. Iceberg clients won't be able to understand the metadata format of the Delta.
- If an Iceberg client has to read the data which is in the delta format, then,

Approach 1 : A copy of the Delta table into Iceberg Format. It will be a rewrite of data and metadata. (Disadvantage : Data Duplication, High Storage Cost, More Write Time).

Approach 2 (Ideal approach) : One copy of Parquet file, which is common for all the formats. There are different copies of Metadata maintained for different formats. (This approach is optimised in terms of storage cost, Write time, etc)

Reading Delta Tables with Iceberg Clients :

Having different metadata files and one common data file underneath, this functionality was previously termed as Delta Lake Universal Format (UniForm) - Multiple formats and single copy of Data with multiple metadata for the respective formats.

Example Demo :

- Create a Notebook and connect to a serverless instance.
- %sql

  use catalog deltalake_catalog;

- Create a table. Set Table properties to be able to read from Iceberg Client.

  %sql

  DROP TABLE IF EXISTS orders_managed;

```
CREATE OR REPLACE TABLE orders_managed (

order_id BIGINT,

sku STRING,

product_name STRING,

product_category STRING,

qty INT,

unit_price DECIMAL(10,2)

) TBLPROPERTIES (‘delta.columnMapping.mode’ = ‘name’,

‘delta.enableIcebergCompactV2’ = ‘true’,

‘delta.universalFormat.enabledFormats’ = ‘iceberg’
```

- Insert records into the table
- When you check the properties of the table using describe table, you will see an entry of “# Delta Uniform Iceberg”. Another metadata file is created for Iceberg format in the storage location.
- Instead of having multiple copies of the data. UniForm approach is more efficient where only one copy of the data is maintained.

Handling Small Files

Handling Small Files

- Consider there 10000 files, each file is of roughly 1mb.

Suppose the following query is executed on the above data-

Select avg(sales) from orders_managed

Note : The overhead involved in handling these large numbers of small files (not adequate sized files) is more than the actual processing.

- However, if we consider 100 files of 100mb each, then these are considerably adequate sized files.
- You have a table and every 1hr there is a new update coming. This will generate new files of a few MBs for every incremental update. Now, this will lead to small file problems. This is a business behavior that after every one hour a new file arrives. This cannot be avoided and how this small file problem can be handled by data engineers.

How to deal with a large number of small files ? Using the **OPTIMIZE** command.

How to execute Manual Optimize -

- Connect to serverless
- Create a table and while creating the table, the auto optimize should be disabled.

  TBLPROPERTIES (

  delta.autoOptimize.optimizeWrite = false,

  delta.autoOptimize.autoCompact = false

)

- Check the properties set by using the "describe detail table" command
- Small files are created when there are multiple inserts. Code to mimic the multiple inserts.
- The parquet file maintains the stats for each file (like the min & max values, etc). This will help in faster retrieval of the required records.

  (Note : The "show performance" option after executing the query, gives the details of the execution. Like - files read, files pruned, etc

- Check the history by executing the following command

  DESCRIBE HISTORY orders_managed

- On Running the “Optimize” combine, it will combine the smaller files into a larger file.

  %sql

  OPTIMIZE orders_managed;

  (It will take all the small files and combine the files to a larger file)

- **What is the best file size for Manual Optimize?**

  Ideal file sizes are - 16MB to 1GB

- spark.databricks.delta.optimize.maxFileSize ~ 1GB (implies that the target files should be less than 1GB)
- We cannot make the changes in case of serverless but can be done on a classic compute.
- **Bin Packing Algorithm**

  100mb, 300mb, 300mb, 100mb, 600mb, 300mb

  Sort the above in descending order

  600mb, 300mb, 300mb, 300mb, 100mb, 100mb

  Distribution into the Bins

  1. **Bin-1 : 600mb, 300mb, 100mb**
  2. **Bin-2 : 300mb, 300mb, 100mb**

Auto Optimize

Auto Optimize

By default Auto Optimize is enabled. On inserting the records, internally optimize will be executed after a certain number of files.

The triggering point for auto optimize to run depends on the value set for the parameters

“spark.databricks.delta.autoCompact.minNumFiles” - Minimum files present to trigger the auto optimize. (Triggering Point)

“spark.databricks.delta.autoCompact.maxNumFiles” - 1GB

Both of the above settings can be changed only in case of classic compute and not in sreverless.

DeltaLake Data Skipping & Partitioning

DeltaLake Data Skipping & Partitioning

Data skipping or file skipping

- When there are more files, then more computational resources are required. The fewer the number of files the fewer resources are required and it is more efficient. Therefore, there arises a need to prune or skip the files that are not relevant.
- How to achieve data skipping? With the help of **Data Layout (Underlying structuring of the data)**
- This can be achieved with the help of **Partitioning / Z-ordering.**
- How does **Partitioning** help in improvising the Data Layout so that the irrelevant data is skipped.
- Consider an example of a Sales table

  If we execute a query with a filter like -

  select * from sales where country = India

  We might have 50 such folders, each folder representing a country

  Country = India

  Country = USA

  Country = Australia

  .

  .

  (For the above query if the files belonging to country India are segregated into a single folder, then the amount of data to be scanned will be drastically reduced. In this case all of the files belonging to India is added into one single folder)

- Laying out the data in a proper folder structure for easier and faster access. Also termed as '**Hive Style Partitioning'**

Example Demo :

- Create a normal table named orders_managed without any partitions and insert the data.

- Create a table with Partition

```
%sql
CREATE TABLE IF NOT EXISTS orders_managed_partitioned (
        order_id BIGINT,
        sku STRING,
        product_name STRING,
        product_category STRING,
        qty INT,
        unit_price DECIMAL(10,2),
        country STRING
)
USING DELTA
PARTITIONED BY (country);
```

- Copy the data from normal to the partitioned table.

```
%sql
insert into orders_managed_partitioned
select * from orders_managed;
```

- This partitioning will be beneficial if in the predicate there is a filter clause on the country column.

  %sql

  select product_category, sum(qty) as total_qty

  from orders_managed_partitioned

  where country = ‘IN’

  group by product_category;

  (It will skip all the other country folders and searches only the IN folder)

  This is the benefit of partitioning, that the data is laid out in a way that it is possible to skip or prune the files that are not relevant.

- Partitioning is only helpful when the reading pattern is based on a column on which the data is partitioned. Partitioning is of no help, if the querying is based on a column other than the partitioned column. In this case, all of the folders or files has to be searched

Challenges with Partitioning :

- Partitioning can be applied only on the columns with low cardinality.
- With File pruning, there are no obvious benefits for data skipping.
- In case of usage pattern changes, the partitioning needs to be changed and that involves a good amount of work involved in rewriting the whole table.
- Can lead to small file problem (Ex: Some countries might have considerably more data compared to the other countries leading to small file problem)
- There are high chances of Data skewness.

(Note: Partitioning works on columns with low cardinality. How to work with columns with high cardinality. Example : column ‘unit-price’ has many distinct values i.e., having high cardinality)

Z-Ordering

Z-Ordering

- Partitioning changes the layout of the data allowing it to skip or prune irrelevant data. Partitioning works on columns where cardinality is low.
- In cases where there are a large number of distinct values where the cardinality is high, then Zoerdering is a better choice.
- **Z ordering** co-locates similar data in the same files.
- Example : Consider a scenario of trip distance which is available in multiple files, Like -

  File 1 - 0, 500

  File 2 - 2, 400

  File 3 - 10, 450

  File 4 - 5, 350

  File 5 - 20, 100

  Say for instance we need the records with trip distances between 50 & 100

  Trip distance > 50 & trip distance < 100

  In case of z-ordered files, the data can be laid out as follows -

  File 1 - 0, 30

  File 2 - 31, 80

  File 3 - 81, 200

  File 4 - 201, 400

  File 5 - 401, 500

- **Z-ordering works by sorting the data and minimizes the overlap.**
- Example Demo :
  1. Pick 10 parquet files from the storage location
  2. Create a Dataframe
  3. Write it to a table by generating 200 (paquet) files

Liquid Clustering

Liquid Clustering

Challenges of Partitioning and Z-ordering :

- There is a struggle involved in determining the **partitioning or the z-order columns.**
- If the reading pattern changes, then all the data needs to be re-written.

Therefore, Liquid clustering came into picture to overcome the above challenges.

- Example scenario :

  Consider we have several files for different years as shown below

  2020 - Small

  2021 - Small

  2022 - Large

  2023 - Medium

  2024 - Small

  2025 - Small

  (Assume - Medium is the ideal size to be processed), then

  2020 & 2021 will be combined to get to the ideal size

  The large file of 2022 could be split into 2 files : 2022-1 & 2022-2

  The 2023 - medium size is retained as it is an ideal size

  2024 & 2025 could be combined together to form a medium sized file.

  (This is done to get ideal sized files)

- Liquid clustering works on only the newly arriving data without having to rework on the older data.
- It is the best replacement for partitioning and z-ordering.
- How to enable/ implement liquid clustering

  %sql

  ALTER TABLE <table-name> CLUSTER BY (<column-name>)

- To remove Liquid clustering

  %sql

  ALTER TABLE <table-name> CLUSTER BY NONE